What Python Call Graphs Miss: Implicit Calls and Shared Names, Not Reflection

Adarsha S P, Sankalp Shankar, Surabhi Singh, Pravith S Upadhyaya, Abhishek Kumar
Dept. of AI & ML, Acharya Institute of Technology, Bangalore, India
{adarsha2966, sankalps.23.beai, surabhia.23.beai, praviths.23.beai, abhishekm.23.beai}@acharya.ac.in

***Abstract*—Python call-graph generators are evaluated mostly on micro-benchmarks. We run PyCG and a Pyright-based extractor on 105 pip-installable GitHub repositories and, where the tests could be traced, score them and three open-weight LLMs against the calls the tests make between the project's own functions. PyCG failed on 22% of the 100 repositories it was run on. Across ten hash seeds its call graph changed on 21 of 77 repositories, moving its recall by up to 0.10 (median 0.00). On 39 traced repositories, the union of both analysers finds a median of 0.69 of observed calls (0.54 pooled). Of the calls it misses, 57% reach functions the caller never names (special methods, library callbacks) and 30% go through shared method names; reflection accounts for 2%. Pyright beats PyCG in 33 of 39 repositories. LLM resolvers, choosing among the names on a caller's executed lines in a sample enriched for hard calls, raise recall from 0.65 to 0.79–0.87 but lower precision from 0.83 to 0.60–0.70. Combining static and LLM answers lifts F1 on unseen repositories only from 0.73 to 0.76, no more than a plain union of Pyright and the best LLM. We will release the corpus, pipelines and measurements.**

***Index Terms*—call graphs, static analysis, dynamic analysis, Python, large language models, empirical study**

I. Introduction

A call graph records which functions call which. Tools for program comprehension, refactoring, dead-code detection and vulnerability reachability are built on one, and code-graph retrieval for LLM agents now uses it to decide which code a model sees [1]. Python makes call graphs hard to build statically: dispatch depends on runtime types, functions are passed around as values, and decorators, properties and registries hide calls behind syntax that does not look like a call.

PyCG [2] is the standard Python call-graph generator and the baseline in recent benchmarks [3] and in studies of LLMs as static analysers [4]. Its accuracy, like that of its successors [5], is mostly reported on hand-written micro-benchmarks, where every call has a known answer. Real repositories are larger, messier and have no answer key. For Java, executing programs to bound the recall of static call graphs is established practice [6], [7]; for Python, DyPyBench compared PyCG with dynamic call graphs on 50 projects and found only 49% of the dynamic edges in PyCG's output [3], but attributed 59% of the missing callees to methods of built-in types and about 12% to naming differences, rather than to analysis gaps. We exclude built-in calls by construction and map every tool onto one node set (Section III-B).

This paper asks what static analysers miss on real Python code once those artifacts are removed, whether the standard tool is a stable instrument, and whether LLMs recover the misses. Our numbers are not a correction of DyPyBench's: the corpus, the tracer and the edge definition all differ. We ask four questions. **RQ1 (reliability):** Does a widely used analyser produce a call graph at all, and the same one twice? **RQ2 (agreement):** How much do two static analysers agree on the same code? **RQ3 (recall):** Of the calls a project's tests make, how many do static analysers find, and where do they miss? **RQ4 (LLMs):** Can LLMs resolve the calls static analysis misses, and at what cost in false claims?

Our findings and contributions are:

1) **PyCG is not a fixed instrument.** The baseline of recent Python call-graph studies fails on 22% of the repositories it was run on (timeouts, crashes, out of memory), and on the timed-out repositories we probed its fixpoint diverges. Where it finishes, the size of its raw output changes with `PYTHONHASHSEED` on 46% of repositories. Comparing exact edge sets over ten seeds, the repository's own call graph changes on 21 of 77 repositories, and PyCG's recall against observed calls moves by up to 0.10 (`term-image`: 0.34–0.43).
2) **Implicit calls and shared names, not reflection.** With calls into built-in types excluded and every tool mapped onto one node set, the union of two static analysers still misses a median of 31% of observed calls per repository. Of all the calls it misses, 57% reach a function the caller never names (special methods, library callbacks) and 30% go through method names shared by several functions. Functions that use reflection account for 2% of misses, and a dynamism score built on reflective and dynamic constructs shows no clear relation to recall ($\rho = -0.23$, $p = 0.156$).
3) **Pyright beats PyCG as a call resolver.** Pyright finds more observed calls than PyCG in 33 of 39 repositories (median per-repository gain 0.08, 95% CI [0.06, 0.15]), at similar precision (0.86 against 0.87), and on 30 of 37 when PyCG's names are mapped leniently.
4) **LLMs reverse the micro-benchmark verdict on recall, not on precision.** Asked which of the names on a caller's executed lines it called, LLMs find more observed calls than static analysis (0.79–0.87 against 0.65) but claim calls the tests never made. A static-plus-LLM combiner trained on other projects gives a small F1 gain on unseen repositories, from 0.73 to 0.76 (paired

TABLE I
THE CLOSEST PYTHON STUDIES AND THIS ONE

	DyPyBench [3]	Venkatesh et al. [4]	This paper
Code	50 real projects	micro-benchmark	105 real repos, 39 traced
Static tools	PyCG	PyCG	PyCG, Pyright
Seed determinism	not reported	not reported	3 seeds on all, 10 on 78
Built-in and naming artifacts	reported after measuring	not applicable	excluded by construction
Misses attributed to	built-in types, naming, `super()`	benchmark categories	implicit calls, shared method names
LLMs scored on execution	—	no	yes

95% CI of the gain [0.00, 0.07]), about as much as a plain union of Pyright and the best LLM.

5) **A pinned corpus and pipeline**, to be released: discovery, dynamism scoring, automated build triage with recorded image digests and dependency locks, traces and every measurement.

II. Related Work

Python call graphs. PyCG [2] computes an assignment graph over modules with a fixpoint and reports high precision and moderate recall on its micro-benchmark. Jarvis [5] adds flow sensitivity and reports higher precision, recall and speed than PyCG. Both evaluate primarily on micro-benchmarks and a handful of applications. We use PyCG because it remains the most common baseline, and we report its failure modes at scale.

Static versus dynamic call graphs. Sui et al. [6] and Helm et al. [7] execute Java programs to bound the recall of static call graphs and attribute misses to language features; Reif et al. [8] test which unsound features each Java framework handles. DyPyBench [3] provides 50 executable Python projects and compares PyCG to dynamic call graphs recorded with DynaPyt [9].

Non-determinism in static analysers. Miao et al. [10] study non-deterministic behaviour in twelve static analysers, PyCG among them, and trace most of it to concurrency, analysis bugs and assumed orderings of unordered data structures. We measure how often hash-seed order changes PyCG's call graph across a corpus, and what it does to recall.

LLMs as static analysers. Venkatesh et al. [4] evaluate 24 LLMs on call-graph and type-inference micro-benchmarks and find PyCG ahead of every LLM on call graphs. We revisit the question on real code with execution-derived labels. Table I sets this study against the two closest Python studies.

III. Study Design

A. Corpus

We searched GitHub for Python repositories with at least 50 stars, a push after August 2023, a `pyproject.toml` or `setup.py`, and a test suite. From the 98,416 matches we drew a seeded random sample (not ranked by stars), froze each repository's commit on the day of discovery, and scored its source with an AST-based dynamism score: weighted counts of dynamic constructs (`getattr` with a non-literal name, dynamic imports, `eval`/`exec`, registries, decorators) per thousand lines, with thresholds calibrated on a 14-repository reference set (LOW below 4.0, HIGH from 10.0). Repositories were binned LOW, MED or HIGH and interleaved into a 558-repository queue, so that stopping early still leaves balanced strata.

An automated triage worked the first 240 repositories of the queue. Each was installed into a pinned `python:3.12` image (recorded by digest), falling back to 3.11 and 3.10 on interpreter or dependency errors, and its tests were collected with `pytest`. Every attempt records the resolved commit, the image digest and a hash of the frozen dependency list. Of these, 118 built and collected at least one test, 64 built but collected none, and the rest did not build. The 105 of the 118 whose licence permits redistribution entered static extraction.

B. Nodes and edges

Both analysers are mapped onto one node set, inventoried from the AST: modules, classes' bodies, functions, methods and lambdas, identified by file path and qualified name (line numbers are not part of identity). Comprehensions fold into their enclosing function; a method call resolves to the defining class in the method resolution order. An edge is *in-universe* when both caller and callee are callables of the repository itself; calls into builtins, the standard library and third-party packages are excluded, because no call-graph tool claims to model them and including them would inflate every miss rate. Because both analysers and the tracer are mapped onto the same node identities, naming differences between tools, which DyPyBench reports [3], can enter only as mapping losses (Section VI).

C. Static analysers

PyCG 0.0.8 runs three times per repository, with `PYTHONHASHSEED` set to 0, 1 and 42. Each run uses its own container, an 8 GB memory cap and a 1800 s timeout, with the repository mounted read-only at the same fixed path, so the hash seed is the only input that changes between runs. The PyCG graph of a repository is the union of the seeds that finished.

Pyright 1.1.411 [11] has no batch call-graph mode. For every call site in the AST we ask `pyright-langserver` for the definition of the callee identifier and map the answer back to a node, so it reports only edges written as calls: a name used without call syntax, such as a property read, is outside its reach. It runs without the repository's dependencies installed. Below, *Pyright* means this extractor.

D. Runtime ground truth

For RQ3 and RQ4 we drew repositories from the 78 where both analysers finished, in two rounds whose rules were written down before either round was traced. The first round

took 20 repositories (8 HIGH, 7 MED, 5 LOW) admitted on Python 3.12 with at least 50 tests, a single top-level package, no heavy native stack and no external service. The second took the 20 remaining repositories with at least 20 tests and a single package, allowing native stacks, except two that need root or a live service. Of these, 3 could not be traced (Section IV-C). Where PyCG is seed-dependent, its graph here is the union of the three seeds of Section III-C. Each repository's own test suite runs under a tracer built on Python 3.12's `sys.monitoring` [12]. It records every call between callables of the package, with the line it came from; calls from test code are attributed to a synthetic entry node, and a call that passes through library code (a callback handed to the standard library) is charged to the nearest package frame. We call these *observed calls*: they cover what one run of the tests executes and nothing else.

E. Call-resolution items

An *item* is a pair (caller, candidate) in which the candidate's name appears on a line of the caller that executed under the tests: as a call, as a bare reference (a property read, a function passed on) or as a class name (reaching its constructor). The label is whether the runtime call graph contains the edge caller→candidate: a property read counts through the getter it invokes, a class name through its `__init__`, and a bare reference only if a frame of this caller, or a library frame it called, made the call. Static tools are scored by the same edge test. For a uniquely named candidate on a line that executed, an unobserved claim is strong evidence of a false claim; for a shared name it is weaker, since another run could dispatch to it, so we report precision separately for the two. A resolver answers yes or no for each item: *name match* (every candidate is a call, as a text search would assume), PyCG, Pyright, their union, and the LLMs. Calls whose target never appears by name in the caller cannot be items; they are counted separately so they do not disappear from anyone's recall.

F. LLM resolvers and a combiner

We query gpt-oss-20b, gpt-oss-120b and qwen3.8-27b, open-weight models served by Groq, at temperature 0. Each prompt shows the caller's whole function body (its own lines, without nested definitions), the definitions of its candidates and the class hierarchy among them, and a JSON schema restricts the answer to candidate identifiers. The candidate list holds only names on lines that executed, the same filter that defines items; the models never see static analysis output. Requests use temperature 0, seed 0 and reasoning effort *low*; each answer records the requested model ID, token usage and the raw response. Because of API rate limits, the LLMs answer a seeded sample of callers that contain at least one hard item (a name shared by several functions, a bare reference, or a pair the static analysers get wrong); every resolver is scored on the same sample. Each corpus repository contributes at most 25 sampled callers; `click` and `toolz`, traced during development, contribute 140 and 60.

A logistic-regression combiner uses ten features: each static analyser's and each LLM's answer, how the name is written (call, reference, class name), whether it is shared by several functions, and whether the candidate is a method of the caller's own class. There is no preprocessing or feature selection, and the hyperparameters (L2 penalty 0.5, 3000 epochs, learning rate 0.3) were fixed while developing on `click` and `toolz`, before any corpus repository was traced. It is scored out-of-fold, once with folds grouped by caller function, so that no function's items are scored by a model trained on them, and once leaving out one whole repository at a time. Leave-one-repository-out is the primary result.

IV. Results

A. RQ1: Reliability of PyCG

Pyright finished every repository whose package layout was detected. Of the 100 repositories whose layout was detected, PyCG failed on 22 (22%): 15 hit the 1800 s timeout on every seed, 4 crashed on every seed (an `IndexError` while reading default arguments, or an `AttributeError`, both inside PyCG), 1 exhausted 8 GB, and on 2 the failure mode itself changed with the seed (a timeout on one seed, a crash or out-of-memory on another). Counted per run, 23% of (repository, seed) runs failed, and 24 repositories failed on at least one seed. When PyCG finishes it is fast (median 0.7 s per run, maximum 582 s, well inside the budget). Size is part of the explanation, since failed repositories are larger (median 935 callable nodes against 181), but not all of it: `chevron`, with 16 callable nodes, timed out on every seed. The failure rate matches DyPyBench, where PyCG produced a call graph for 39 of 50 projects [3].

On 36 of the 78 repositories where PyCG finished (46%), its raw output differed in size between the three seeds. For the raw output this count-based check is a lower bound: two runs can return different edges in equal numbers. Because the counts of unique edges differ, output order or serialisation cannot explain it, and repository path, image and tool version were identical across runs, so the seed is the only input that differed. Seeds 0 and 1 ran on both of our hosts: termination agreed, hangs included, in 156 of 156 runs; on the 36 seed-dependent repositories the same seed produced the same output size in 70 of 70 runs, and on the 40 seed-independent ones identical edge sets. The spread, $(\max - \min)/\max$ of the raw edge counts, is usually small (median 1.0%) but has a long tail: 7 repositories vary by 10% or more, and on `movingpandas` the edge count varies by 65% between seeds. It occurs in every stratum. A tool built on PyCG pins the seed, noting that it shifts PyCG's fixpoint [13], and Miao et al. include PyCG in a study of non-deterministic analysers [10]; how often the seed changes its call graph on real repositories, and by how much recall, had not been measured.

Mechanism. PyCG computes its graph with a fixpoint whose passes walk hash-ordered sets, and runs it with no iteration cap; set iteration order depends on `PYTHONHASHSEED`, which is random per process unless pinned. The seed therefore changes which fixpoint is reached, not only the order of

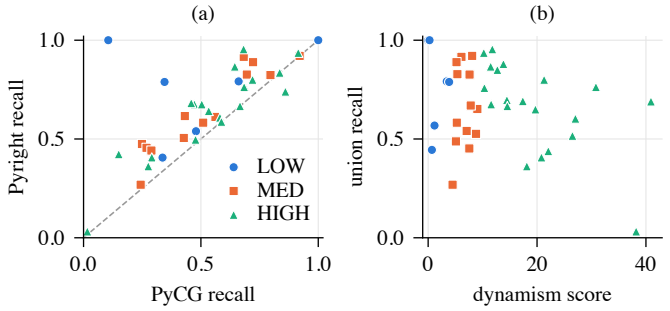


Fig. 1. Recall against observed calls, one point per repository. (a) Pyright against PyCG; points above the diagonal are repositories where Pyright finds more. (b) Union recall against the dynamism score used to stratify the corpus (Spearman $\rho = -0.23$, $p = 0.156$, $n = 39$).

the output. Comparing exact in-universe edge sets over ten seeds, the repository's own call graph changed on 21 of 77 repositories (`deepblast` finished only one seed). Most changes are small: the median of each repository's minimum pairwise Jaccard is 0.98 (IQR 0.94–1.00), 3 fall below 0.9, and `python-lazy-object-proxy` reaches 0.40. The count check misleads both ways: 16 of the 35 it flagged kept identical in-universe edges, and 2 it passed did change.

Two probed timeouts do not converge. Replaying PyCG's own loop on `chevron` (16 callable nodes) with an iteration cap and a log per pass, its abstract state grew by 12 definitions on every pass, from 177 to 1420 over 100 passes, without converging or revisiting a state: each pass adds definitions under a freshly numbered lambda (`render.<lambda192>` by pass 50), so the convergence check can never succeed. On `muscriptor` the state exploded instead: the names it tracks grew from 555 to 18,387 in 4 passes as attributes accumulated on imported names, and the last pass alone took 22 minutes. We did not probe the other timeouts. Termination itself can depend on the seed: in the three-seed run the failure mode changed with the seed on 2 repositories, and in the ten-seed sweep 4 repositories finished on some seeds and ran past 30 minutes on others (e.g., `CorridorKey`).

B. RQ2: Agreement between static analysers

On the 78 repositories where both finished, the Jaccard similarity of the two analysers' in-universe edge sets has a median of 0.65 (interquartile range 0.56–0.80); 16 repositories fall below 0.5 and 19 reach 0.8 or more. Of the 33,368 edges found by either tool, 11% are found only by PyCG and 27% only by Pyright. Two analysers reading identical source disagree on roughly a third of the call graph; deciding which is right needs the runtime labels of RQ3.

C. RQ3: Static recall against runtime calls

We traced the test suites of the sampled repositories, plus `click` and `toolz` from the dynamism score's reference set. Of these, 3 could not be measured: `kolla-ansible`'s tests never import its own package, `wsgidav`'s suite produced no countable test results under our runner, and `recurrentgemma` keeps its tests beside the code (`foo_test.py`), which our runner cannot separate from package code. Across the remaining 39 repositories the tests exercised 10,882 distinct calls between functions and methods of the packages: 7,282 whose callee the caller names, less 59 recursive self-calls, plus 3,659 whose callee it never names. Fig. 1 shows each repository; per-repository values are in the released data.

TABLE II
EVERY OBSERVED CALL THE STATIC UNION MISSES (4,953), BY HOW THE CALLEE APPEARS IN THE CALLING FUNCTION. CLASSES ARE EXCLUSIVE. POOLED: SHARE OF ALL MISSES, ROUNDED; MEDIAN: PER-REPOSITORY SHARE OF EACH CLASS.

Callee in the calling function	Pooled	Median
not named, reached through library code	34%	32%
not named, special method	22%	13%
call, shared name	23%	6%
call, unique name	3%	0%
reference, shared name	6%	0%
reference, unique name	10%	11%
class name	1%	0%

How much is missing. The union of both static graphs contains a median of 0.69 of the observed calls per repository (95% CI [0.60, 0.79]), from 0.03 to 1.00. Pooled over all 10,882 observed calls, so that large repositories weigh more, it holds 0.54 (PyCG 0.39, Pyright 0.52). Without the 3 repositories with fewer than 20 observed calls the medians barely move (union 0.68, Pyright 0.66, PyCG 0.52). Pyright alone reaches 0.68 and PyCG 0.53; Pyright finds more of the observed calls in 33 of 39 repositories, ties on 5 and is behind on 1 (Fig. 1a; median per-repository gain 0.08, 95% CI [0.06, 0.15], Wilcoxon $p < 0.001$), so in this corpus the type checker is the stronger call-graph source. Pooled over 33,020 call-resolution items (7,282 of them observed calls), the union has precision 0.85 and recall 0.70 (PyCG 0.87/0.51, Pyright 0.86/0.69). Plain name matching, which treats every name on an executed line as a call, has precision 0.22: most names in code that ran are not calls the tests made. Union precision is 0.88 on calls to uniquely named functions and 0.73 on shared names, where an unobserved claim may be a call that another run would make.

Where it misses. Table II splits the 4,953 observed calls the union misses. Most (57%) reach a function the caller never names: special methods invoked implicitly (operators, iteration, attribute hooks) and callbacks or functions reached through library code. No call site in the caller names these callees, so a resolver of call sites, as our Pyright extractor and the LLMs are, cannot report them directly; the union still holds 23% of them. A resolver that modelled operator dispatch and library callbacks could recover more. They explain the lowest recall: `python-lazy-object-proxy` (0.03) is a proxy-object library whose tests exercise operator overloading, so almost every observed call is a special method that no call site names. Method names shared by several functions come next, with 30% of all misses and 69% of those whose callee the caller names: dispatch over a class hierarchy or over unrelated

classes with a common method name. `ShivyC`, a C compiler, defines `make_il` on 29 syntax-tree classes and `make_asm` on 20, and the union holds fewer than half of its observed calls. Shared-name misses concentrate in a few large repositories: the median repository loses 6% of its misses to shared-name calls but 57% to unnamed ones taken together. Names written without call syntax, such as property reads and functions passed as values, account for 17% of misses.

Not reflection. Functions that use reflection (a `getattr`-family call with a computed name, `eval` or `exec`, a dynamic import) make only 2% of observed calls (213). Their calls are missed at a rate of 0.50 against 0.45 elsewhere, and they account for 2% of all misses. Across repositories, the dynamism score used to stratify the corpus shows no evidence of a relation to recall (Spearman $\rho = -0.230$, $p = 0.156$; Fig. 1b; without the 3 repositories with fewer than 20 observed calls, $\rho = -0.09$, $p = 0.604$), and recall does not differ by stratum (Kruskal–Wallis $p = 0.590$; medians 0.79 (n=6) for LOW, 0.65 (n=13) for MED and 0.69 (n=20) for HIGH). Static analysers lose calls in this corpus to implicit calls and dispatch over shared names, not to reflection.

Coverage and stability of the labels. The tests executed a median of 67% of each package's statement lines (24% to 98%). Recall falls as coverage rises, a marginal relation (Spearman $\rho = -0.33$, $p = 0.046$, 95% CI $[-0.58, 0.00]$), and shows no clear relation to the number of tests ($\rho = -0.231$, $p = 0.158$). If suites that reach more code also reach more hard calls, recall on lightly tested repositories is overstated. Tracing every suite a second time on 37 repositories, the observed-call sets of the two runs had a median Jaccard similarity of 1.00 (minimum 0.99).

Does the seed change recall? Across ten seeds, PyCG's recall against observed calls changed on 8 of the 37 traced repositories in the sweep (`click` and `toolz` are outside it), by up to 0.10 (`term-image`: 0.34–0.43; median 0.00), while the union's moved by at most 0.02: Pyright holds most of the edges PyCG gains or loses. On 8 repositories some of the edges that change with the seed are observed calls.

D. RQ4: LLMs as call resolvers

On the sampled callers, which are enriched for hard items, every LLM finds more of the observed calls than static analysis does: recall rises from 0.65 for the static union on the same items to 0.83 for gpt-oss-20b, 0.87 for gpt-oss-120b and 0.79 for qwen3.8-27b. Precision falls from 0.83 to between 0.60 and 0.70: the models claim calls the tests never made. The larger gpt-oss model has the highest recall and the lowest precision. On real code, then, the micro-benchmark finding that PyCG outperforms LLMs [4] holds for precision but reverses for recall, on a narrower task: our models choose among names on executed lines rather than build a graph. With whole repositories resampled, the 95% interval for recall is [0.78, 0.95] for gpt-oss-120b against [0.55, 0.74] for the static union, but the F1 intervals overlap ([0.64, 0.77] against [0.66, 0.78]), and an LLM has the higher F1 in only 15–18 of 39 repositories. Calls the caller never names are not items, so this comparison covers only named calls. The sample holds 940 callers, and 24% of its items are observed calls. Only 53 callers entered it solely through a static error; without them, static precision and recall are 0.88 and 0.64 and the LLMs' 0.61–0.73 and 0.78–0.87, so the comparison does not rest on that criterion.

TABLE III
CALL RESOLUTION ON THE ITEMS EVERY LLM ANSWERED, SCORED LEAVING ONE REPOSITORY OUT. LAST COLUMN: THE LEARNED COMBINER'S F1 MINUS THIS ROW'S, 95% PAIRED REPOSITORY-BOOTSTRAP INTERVAL.

Resolver	P	R	F1	Combiner − row
PyCG	0.84	0.49	0.62	[0.08, 0.19]
Pyright	0.83	0.64	0.73	[0.00, 0.07]
PyCG ∪ Pyright	0.83	0.65	0.73	[0.00, 0.07]
gpt-oss-20b	0.63	0.83	0.71	[0.00, 0.11]
gpt-oss-120b	0.60	0.87	0.71	[0.01, 0.11]
qwen3.8-27b	0.70	0.79	0.74	[−0.02, 0.06]
majority of the three LLMs	0.66	0.85	0.74	[−0.02, 0.08]
Pyright ∪ qwen3.8-27b	0.70	0.81	0.75	[−0.03, 0.06]
static ∪ any LLM	0.54	0.91	0.68	[0.03, 0.16]
learned combiner	0.80	0.73	0.76	

The LLMs complement static analysis rather than duplicate it: on observed calls the static union missed they recover 0.46–0.70, and on those it found they agree on 0.94–0.97. The gain comes from hard names. Every resolver finds almost all calls to uniquely named functions. On shared method names static recall is 0.44 against 0.52–0.75 for the LLMs. Names written without call syntax are almost absent from static graphs (recall at most 0.05); the LLMs recover them at 0.74–0.86, but with precision of only 0.32–0.43.

Table III scores every resolver on the items all three LLMs answered, leaving one whole repository out at a time. Adding every LLM claim to the static graph maximises recall at a large cost in precision. A majority vote of the three LLMs matches the static union on F1 (0.74 against 0.73). The learned combiner, trained on 5,573 items from 39 repositories, gives every source a positive weight and discounts names written without call syntax; on a repository it has never seen it reaches F1 0.76 (precision 0.80, recall 0.73, against 0.83 and 0.65 for the union). It beats the union on 20 of 39 repositories and loses on 9. Paired over repositories, the 95% CI of its F1 gain is [0.00, 0.07] over the static union and [0.00, 0.07] over Pyright alone: a small gain whose interval reaches zero. Over the best single LLM (qwen3.8-27b), the majority vote and the union of Pyright with qwen3.8-27b its gain is indistinguishable from zero: a plain union does about as well as the learned filter. Without `click` and `toolz`, on which its hyperparameters were set, it scores 0.75 against 0.70 for the union (difference [0.01, 0.09]). With folds grouped by caller, which lets it see the test project's other functions, it reaches 0.75, no better.

V. DISCUSSION

Use a type checker as the baseline. Pyright alone holds nearly all of the union (median recall 0.68 against 0.69), and adding it damps PyCG's seed effect on recall to at most

0.02. Tools that consume call graphs should start from a type checker, not from PyCG; studies that keep PyCG should pin and report `PYTHONHASHSEED` or report the spread over several seeds.

LLMs are recall tools, not replacements. Static analysis is precise but blind to dispatch over shared names and to references; the LLMs see those but over-claim, and neither sees implicit calls, the largest class of misses. Neither is enough alone; combined, whether by a learned filter or by a plain union of Pyright and the best LLM, they give a small F1 gain on unseen repositories (0.73 to 0.76). A tool that consumes call graphs can treat LLM answers as a second vote, not as a replacement.

VI. Threats to Validity

Ground truth is workload-conditional. The tracer sees only what the test suites execute. Recall is measured against the calls the tests made, a subset of the calls the program can make, and precision only on lines that executed; a static edge that no test exercised is neither confirmed nor counted against the tool.

Environments. Runtime traces (RQ3, RQ4) were collected on a second x86-64 host, installing each repository by the triage recipe rather than from its frozen dependency list, so dependency versions may differ; the static graphs are computed from source and are unaffected. `click` and `toolz` were traced on an arm64 machine; both are pure Python.

Choice of analysers. PyCG is archived but still the baseline in 2024–25 studies [3], [4], so its seed dependence bears on their numbers. Measuring Jarvis [5] on the same traces is future work.

Tool versions and configuration. We measure one version of each analyser. PyCG's 30-minute budget is a choice; on the two timed-out repositories we probed, the fixpoint did not converge, but we did not probe all 15.

LLM sample and serving. The LLMs answered a sample of callers enriched for hard items, so their scores are not comparable to static scores on all items. One enrichment criterion, a pair the static analysers get wrong, lowers static recall on the sample by construction; without the callers it alone admitted, the results barely change (Section IV-D). The candidate list comes from executed lines, so the LLMs choose among names the tests reached rather than build a graph, which flatters them relative to a deployed tool. The corpus is public and may be in the models' training data. Each model answered once; we record the requested model, but not the serving model. The combiner's hyperparameters were fixed on `click` and `toolz`, which are also evaluated; without them its gain holds (Section IV-D).

Mapping. PyCG's names are mapped onto our node set strictly. Rerun on the 37 traced corpus repositories and remapped leniently (a class to its `__init__`, a subclass method to the base that defines it, a unique name suffix), its median recall rises from 0.53 to 0.58 and its constructor recall from 0.50 to 0.92; 24% of the calls Pyright finds and strict PyCG misses are mapping losses. Pyright still finds more on 30 of 37 repositories, and the union is unchanged (+0.002 pooled).

Corpus. The corpus excludes repositories that cannot be installed from their root with `pip` or that collect no tests, which biases it against applications and monorepos. The runtime subset favours small, pure-Python packages whose test suites need no external services. The LOW stratum of the runtime subset is the thinnest, with 6 repositories.

VII. Conclusion

On pip-installable, tested Python repositories, the standard static call-graph generator fails on a fifth of the corpus, its call graph depends on the hash seed on 21 of the 77 it finished, and two static analysers disagree on a third of the edges. Measured against the calls tests actually make, the union of two static analysers finds a median of 0.69 of the calls per repository (0.54 pooled). Most of what it misses are implicit calls that no call site names, then calls through shared method names; reflection accounts for 2%. LLMs recover many of the named misses at a cost in precision, and combining the two, whether by a learned filter or a plain union, gives only a small F1 gain on unseen repositories (0.73 to 0.76). Corpus pins, image digests, pipelines and all results are available at https://github.com/sankalp-happy/python-callgraph-gaps.

Acknowledgment

LLM queries were made through the co-authors' Groq API accounts, with Groq's permission.

References

[1] S. Ouyang *et al.*, "RepoGraph: Enhancing AI software engineering with repository-level code graph," arXiv:2410.14684, 2024.
[2] V. Salis, T. Sotiropoulos, P. Louridas, D. Spinellis, and D. Mitropoulos, "PyCG: Practical call graph generation in Python," in *Proc. ICSE*, 2021, pp. 1646–1657.
[3] I. Bouzenia, B. P. Krishan, and M. Pradel, "DyPyBench: A benchmark of executable Python software," *Proc. ACM Softw. Eng.*, vol. 1, no. FSE, pp. 338–358, 2024.
[4] A. P. S. Venkatesh, R. Sunil, S. Sabu, A. M. Mir, S. Reis, and E. Bodden, "An empirical study of large language models for type and call graph analysis in Python and JavaScript," *Empir. Softw. Eng.*, vol. 30, no. 6, 2025.
[5] K. Huang, Y. Yan, B. Chen, Z. Tao, and X. Peng, "Scalable and precise application-centered call graph construction for Python," arXiv:2305.05949, 2023.
[6] L. Sui, J. Dietrich, A. Tahir, and G. Fourtounis, "On the recall of static call graph construction in practice," in *Proc. ICSE*, 2020, pp. 1049–1060.
[7] D. Helm, S. Keidel, A. Kampkötter, J. Düsing, T. Roth, B. Hermann, and M. Mezini, "Total recall? How good are static call graphs really?" in *Proc. ISSTA*, 2024.
[8] M. Reif, F. Kübler, M. Eichberg, D. Helm, and M. Mezini, "Judge: Identifying, understanding, and evaluating sources of unsoundness in call graphs," in *Proc. ISSTA*, 2019.
[9] A. Eghbali and M. Pradel, "DynaPyt: A dynamic analysis framework for Python," in *Proc. ESEC/FSE*, 2022, pp. 760–771.
[10] M. Miao, A. Mordahl, D. Soles, A. Beideck, and S. Wei, "An extensive empirical study of nondeterministic behavior in static analysis tools," in *Proc. ICSE*, 2025, pp. 1064–1076.
[11] Microsoft, "Pyright: Static type checker for Python," https://github.com/microsoft/pyright, version 1.1.411.
[12] M. Shannon, "PEP 669 – low impact monitoring for CPython," https://peps.python.org/pep-0669/, 2022.
[13] codellm-devkit, "codeanalyzer-python: pinning PYTHONHASHSEED for PyCG," https://github.com/codellm-devkit/codeanalyzer-python, commit ae2df74.